In [20]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

X_train = pd.read_csv("X_train.csv")
y_train = pd.read_csv("y_train.csv")
X_test = pd.read_csv("X_test.csv")

print("X_train:", X_train.shape)
print("y_train:", y_train.shape)
print("X_test:", X_test.shape)
```

```
X_train: (487680, 13)
y_train: (3810, 3)
X_test: (488448, 13)
```

In [21]:
```
sensor_cols = ['orientation_X', 'orientation_Y', 'orientation_Z', 'orientation
               'angular_velocity_X', 'angular_velocity_Y', 'angular_velocity_Z
               'linear_acceleration_X', 'linear_acceleration_Y', 'linear_accel

def aggregate_features(df):
    grouped = df.groupby("series_id")[sensor_cols]
    feat = pd.concat([
        grouped.mean().add_suffix("_mean"),
        grouped.std().fillna(0).add_suffix("_std"),
        grouped.min().add_suffix("_min"),
        grouped.max().add_suffix("_max")
    ], axis=1).reset_index()
    return feat

X_train_feat = aggregate_features(X_train)
X_test_feat = aggregate_features(X_test)

print("X_train_feat:", X_train_feat.shape)
print("X_test_feat:", X_test_feat.shape)


y_train_agg = y_train.drop_duplicates("series_id").set_index("series_id")

classes = sorted(y_train_agg["surface"].unique())
class_to_id = {c: i for i, c in enumerate(classes)}
id_to_class = {i: c for c, i in class_to_id.items()}

print("Классы:", class_to_id)
```

```
X_train_feat: (3810, 41)
X_test_feat: (3816, 41)
Классы: {'carpet': 0, 'concrete': 1, 'fine_concrete': 2, 'hard_tiles': 3, 'har
d_tiles_large_space': 4, 'soft_pvc': 5, 'soft_tiles': 6, 'tiled': 7, 'wood': 8}
```

In [22]:
```
df_train = X_train_feat.merge(y_train_agg, on="series_id")
df_train["label"] = df_train["surface"].map(class_to_id)

feature_cols = [c for c in df_train.columns if c not in ["series_id", "surface
```

```
X = df_train[feature_cols].fillna(0).values
y = df_train["label"].values

print(f"Размер выборки: {X.shape[0]}")
print(f"Количество признаков: {X.shape[1]}")
print(f"Количество классов: {len(classes)}")

np.random.seed(42)
indices = np.random.permutation(len(X))
split = int(0.8 * len(X))
train_idx, val_idx = indices[:split], indices[split:]

X_tr, X_val = X[train_idx], X[val_idx]
y_tr, y_val = y[train_idx], y[val_idx]

print(f"Тренировка: {X_tr.shape[0]} примеров")
print(f"Валидация: {X_val.shape[0]} примеров")
```

```
Размер выборки: 3810
Количество признаков: 41
Количество классов: 9
Тренировка: 3048 примеров
Валидация: 762 примеров
```

In [23]:

```
def gini(y):
    if len(y) == 0:
        return 0.0
    p = np.bincount(y) / len(y)
    return 1.0 - np.sum(p * p)

class Node:
    __slots__ = ('feat', 'q1', 'q2', 'left', 'mid', 'right', 'pred', 'proba')

    def __init__(self, feat=None, q1=None, q2=None,
                 left=None, mid=None, right=None,
                 pred=None, proba=None):
        self.feat = feat
        self.q1 = q1
        self.q2 = q2
        self.left = left
        self.mid = mid
        self.right = right
        self.pred = pred
        self.proba = proba
class ThreeWayTree:
    def __init__(self, max_depth=8, min_samples=15, feat_sample='sqrt'):
        self.max_depth = max_depth
        self.min_samples = min_samples
        self.feat_sample = feat_sample
        self.n_classes = None
        self.n_feats = None
        self.root = None

    def _sample_features(self):
```

```
        if self.feat_sample == 'sqrt':
            k = int(np.sqrt(self.n_feats))
        else:
            k = self.n_feats
        return np.random.choice(self.n_feats, size=min(k, self.n_feats), repla

    def _split_masks(self, X, feat, q1, q2):
        v = X[:, feat]
        return [(v <= q1), (v > q1) & (v <= q2), (v > q2)]

    def _best_split(self, X, y, feats):
        best_feat = None
        best_q = None
        best_gain = -1
        parent_gini = gini(y)

        for f in feats:
            vals = X[:, f]
            if len(np.unique(vals)) < 3:
                continue

            q1, q2 = np.percentile(vals, [33, 66])
            if q1 >= q2:
                continue

            masks = self._split_masks(X, f, q1, q2)
            parts = [(X[m], y[m]) for m in masks if np.sum(m) > 0]

            if len(parts) < 2:
                continue

            weighted_gini = sum(len(yp) / len(y) * gini(yp) for _, yp in parts
            gain = parent_gini - weighted_gini

            if gain > best_gain:
                best_gain = gain
                best_feat = f
                best_q = (q1, q2)

        return best_feat, best_q

    def _build(self, X, y, depth):
        if depth >= self.max_depth or len(y) < self.min_samples or len(np.unic
            counts = np.bincount(y, minlength=self.n_classes)
            proba = counts / counts.sum()
            return Node(pred=np.argmax(counts), proba=proba)

        feats = self._sample_features()
        feat, q = self._best_split(X, y, feats)

        if feat is None:
            counts = np.bincount(y, minlength=self.n_classes)
            proba = counts / counts.sum()
```

```
            return Node(pred=np.argmax(counts), proba=proba)

        masks = self._split_masks(X, feat, q[0], q[1])
        children = []

        for m in masks:
            if np.sum(m) == 0:
                counts = np.bincount(y, minlength=self.n_classes)
                proba = counts / counts.sum()
                children.append(Node(pred=np.argmax(counts), proba=proba))
            else:
                children.append(self._build(X[m], y[m], depth + 1))

        return Node(feat=feat, q1=q[0], q2=q[1],
                    left=children[0], mid=children[1], right=children[2])

    def fit(self, X, y):
        self.n_classes = len(np.unique(y))
        self.n_feats = X.shape[1]
        self.root = self._build(X, y, 0)

    def _predict_one(self, x, node):
        if node.pred is not None:
            return node.pred
        if x[node.feat] <= node.q1:
            return self._predict_one(x, node.left)
        elif x[node.feat] <= node.q2:
            return self._predict_one(x, node.mid)
        else:
            return self._predict_one(x, node.right)

    def _proba_one(self, x, node):
        if node.proba is not None:
            return node.proba
        if x[node.feat] <= node.q1:
            return self._proba_one(x, node.left)
        elif x[node.feat] <= node.q2:
            return self._proba_one(x, node.mid)
        else:
            return self._proba_one(x, node.right)

    def predict(self, X):
        return np.array([self._predict_one(x, self.root) for x in X])

    def predict_proba(self, X):
        return np.array([self._proba_one(x, self.root) for x in X])
```

In [24]:

```
class RandomForest:

    def __init__(self, n_trees=20, max_depth=8, min_samples=15, feat_sample='s
        self.n_trees = n_trees
        self.max_depth = max_depth
        self.min_samples = min_samples
```

```
        self.feat_sample = feat_sample
        self.trees = []
        self.n_classes = None

    def _bootstrap(self, X, y):
        n = len(X)
        idx = np.random.choice(n, size=n, replace=True)
        return X[idx], y[idx]

    def fit(self, X, y):
        self.n_classes = len(np.unique(y))
        for i in range(self.n_trees):
            Xb, yb = self._bootstrap(X, y)
            tree = ThreeWayTree(
                max_depth=self.max_depth,
                min_samples=self.min_samples,
                feat_sample=self.feat_sample
            )
            tree.fit(Xb, yb)
            self.trees.append(tree)


    def predict(self, X):
        preds = np.array([t.predict(X) for t in self.trees]).T
        return np.array([np.bincount(row).argmax() for row in preds])

    def predict_proba(self, X):
        probas = np.array([t.predict_proba(X) for t in self.trees])
        return np.mean(probas, axis=0)
```

In [25]:
```
rf = RandomForest(
    n_trees=20,
    max_depth=8,
    min_samples=15,
    feat_sample='sqrt'
)

rf.fit(X_tr, y_tr)

y_pred = rf.predict(X_val)
y_proba = rf.predict_proba(X_val)
```

In [26]:
```
def accuracy(y_true, y_pred):
    return np.mean(y_true == y_pred)

def macro_scores(y_true, y_pred, n_classes):
    precisions, recalls, f1s = [], [], []

    for c in range(n_classes):
        tp = np.sum((y_true == c) & (y_pred == c))
        fp = np.sum((y_true != c) & (y_pred == c))
        fn = np.sum((y_true == c) & (y_pred != c))
```

```
        p = tp / (tp + fp) if tp + fp > 0 else 0
        r = tp / (tp + fn) if tp + fn > 0 else 0
        f = 2 * p * r / (p + r) if p + r > 0 else 0

        precisions.append(p)
        recalls.append(r)
        f1s.append(f)

    return np.mean(precisions), np.mean(recalls), np.mean(f1s)

acc = accuracy(y_val, y_pred)
prec, rec, f1 = macro_scores(y_val, y_pred, len(classes))


print(f"Accuracy:  {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall:    {rec:.4f}")
print(f"F1-score:  {f1:.4f}")
```

```
Accuracy:  0.8451
Precision: 0.8661
Recall:    0.7829
F1-score:  0.8044
```

In [27]:

```
def roc_curve_binary(y_true_bin, y_score):
    thresholds = np.sort(np.unique(y_score))[:-1]
    tpr, fpr = [], []
    p = np.sum(y_true_bin == 1)
    n = np.sum(y_true_bin == 0)

    for th in thresholds:
        pred = (y_score >= th).astype(int)
        tp = np.sum((y_true_bin == 1) & (pred == 1))
        fp = np.sum((y_true_bin == 0) & (pred == 1))
        tpr.append(tp / p if p > 0 else 0)
        fpr.append(fp / n if n > 0 else 0)

    return np.array(fpr), np.array(tpr)

plt.figure(figsize=(11, 8))
colors = plt.cm.Set2(np.linspace(0, 1, len(classes)))

for idx, cls in enumerate(classes):
    y_bin = (y_val == idx).astype(int)
    y_sc = y_proba[:, idx]
    fpr, tpr = roc_curve_binary(y_bin, y_sc)

    order = np.argsort(fpr)
    plt.plot(fpr[order], tpr[order], color=colors[idx], lw=2, label=cls)

plt.plot([0, 1], [0, 1], 'k--', label="Случайный (AUC=0.5)")

plt.xlim([-0.02, 1.02])
plt.ylim([-0.02, 1.02])
```

```
plt.xlabel("False Positive Rate (FPR)", fontsize=12)
plt.ylabel("True Positive Rate (TPR)", fontsize=12)
plt.title("ROC-кривые для каждого типа поверхности (One-vs-Rest)", fontsize=14
plt.legend(loc='lower right', fontsize=9)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("roc_curves.png", dpi=150)
plt.show()
```